CPT205 – 2D Greeting Card Project Report

Xi'an Jiaotong–Liverpool University (XJTLU)

Junyan He ID: 2363304

Abstract

This project presents a 2D interactive greeting card celebrating the 20th Anniversary of Xi'an Jiaotong–Liverpool University (XJTLU). Built with OpenGL FreeGLUT, it combines animation, interaction, and visual design to create a playful yet elegant scene. The design focuses on smooth transitions between stages, pixel-style graphics, and layered rendering effects such as soft lighting, dynamic clouds, and fireworks. Through simple controls, users can explore the campus environment and experience a lively celebration atmosphere.

1. Introduction

This project is a 2D interactive greeting card system themed around the 20th Anniversary of Xi'an Jiaotong-Liverpool University (XJTLU). The main objective of the project is to implement a greeting card that integrates animation, interaction, and multi-scene transitions within the FreeGLUT environment.

My card is designed with three main stages (or scenes): the Cover, the Campus, and the Celebration(The viewport in the third scene is larger), which are controlled by the global state variable `gStage` in coordination with time. The following sections first present the README of the system,which includes explanations of some code functions, interaction instructions, and key state transitions, followed by the design and transitions of the three scenes are discussed in detail. Subsequently, the hint system are described. Finally, the report concludes with a summary of the existing issues and potential improvements of the code.

2. System Overview (README)

To provide a quick visual reference of the system layout, figure 1 illustrates the overall composition used in the README.

2.1. Overview

This project is a single-file interactive greeting card built with FreeGLUT + Core OpenGL. It consists of three major scenes:

- **Cover Scene:**The scene shows a bright starburst background, rows of flags and balloons, a cat walking back and forth, and a fish-shaped cursor.
- **Main Scene:**The scene shows a striped gradient sky, flat-topped cloud clusters, campus buildings, a soft flashlight made with five stencil layers, and a cat the player can control.
- **Celebration Scene:** displays fireworks, colorful particle dots, and a double-stroked title text.

The transition between the Cover and Main scenes is implemented with a black fade-in/fade-out effect.

2.2. Code Map (Functions & Data)

Only core interfaces are listed below, for details, refer to functions with the same names in the source code.

- **Config & State:** window size, color palette, camera and light parameters, `AppStage`, `GameState`, input states, and time accumulators.
- **Utils:** helper functions such as `currentOrthoBounds`, `clampf`, `lerp`, `smooth01`, `easeInOut`, `hash`.
- **Primitives:** drawing rectangles, lines, circles, and ellipse fans.
- **Reusable Widgets:** flag banners, buttons, dialogue bubbles, and hint labels.
- **Backdrop & Grass:** `drawSkyGradientAndDots`, `drawGrassTufts`, `drawExtendedBackdrop`.
- **Clouds:** `drawCloudBlob`, `drawCloudCluster`, `drawCloudsClustersFlat` (with stencil clipping and triple-layer transitions).
- **Campus:** stripe blocks, glass connectors, window grids, platforms, main/left/landmark/gym buildings, stone monument + flag combinations.
- **Cat & Lantern:** `drawHero`, `drawLantern`.
- **Soft Lamp:** `writeLampToStencil5` (writes stencil regions); `drawBlackCoversSoft5` (applies layered blackout by stencil).
- **Particles & Fireworks:** `Rocket`, `Particle`, tail emission `emitTrail`, burst emission `emitBurst`, periodic spawning `spawnFireworkBurst`, updating and drawing with `updateParticles` and `drawParticles`.
- **Cover Scene:** `drawCoverBackgroundRadial`, `drawCoverPixelTitle_Trapezoid`, `drawFishCursorAtMouse`, `drawCoverScene`; transition handled in the `COVER` branch of `display()`.
- **Update:** `updateHero`, `updateStateMachine`, `onIdle` (controls cover cat movement, pause, and timing for scene transitions).
- **Display:** routes rendering by `AppStage` and overlays hints, dialogues, and titles.

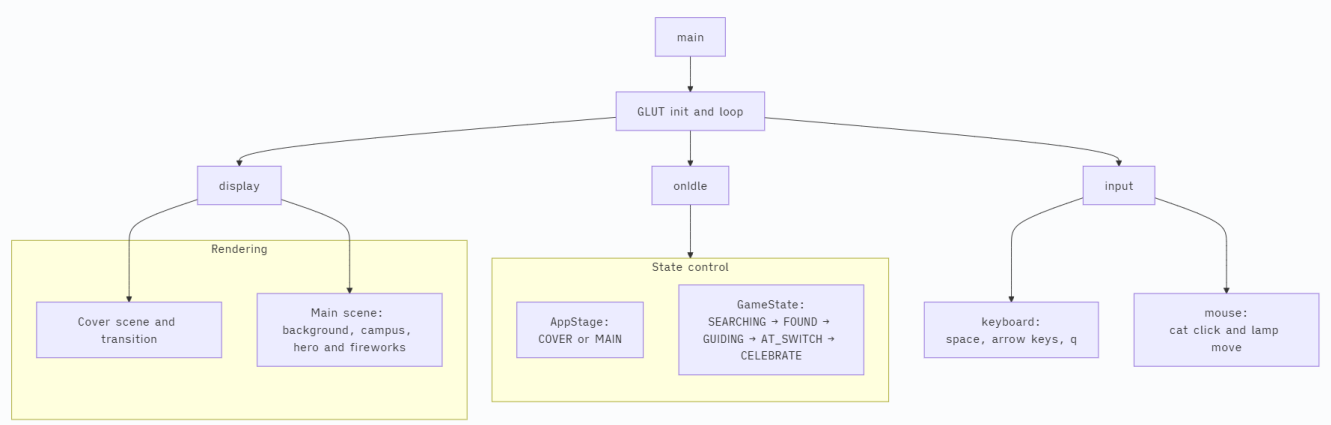


Figure 1 README overview placeholder.

2.3. Interaction & State Machine

Figure 2 and table 1 summarize the control mapping and state progression.

Controls

Table 1 User Controls

Scene	How to Operate
Cover	Left-click — Feed the fish to the cat.
Main	Move the mouse — Shine the light to find the cat. W / A / S / D — Move the character. SPACE — Pull the switch. Q — Exit after celebration.

Cover Scene:

- Left-click: feed the fish treat to the cat (activates the cover hint).

Main Scene:

- Mouse movement (SEARCHING): moves the flashlight center to find the cat.
- W/A/S/D or Arrow keys: control the character in the `GUIDING` or `AT_SWITCH` state.
- SPACE: triggers the switch during `AT_SWITCH`, entering the celebration stage.
- Q: after 5 seconds in celebration, displays the "Press Q to exit " hint.

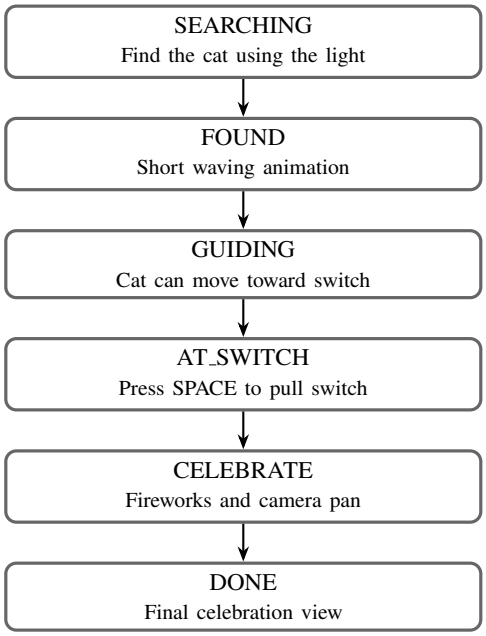
Application Stages

`AppStage:COVER` → `AppStage::MAIN` (black transition lasting about 2 seconds, defined by `COVER_TRANS_TOTAL`).

2.4. Frame Composition

- **COVER Scene:** background (starburst) → flags → pixel title "20th" → cat & balloons → fish cursor → hint overlay → black fade transition.
- **MAIN Scene:** background & grass → clouds → campus → cat & lantern → stencil write → layered blackout (5 layers) → confetti → (if celebrating) fireworks particles → title overlays.

Figure 2 Game State Flow



3. Scene 1: Cover Design Details

The following figure provides the cover layout.



Figure 3 Cover scene layout placeholder.

As shown in the figure 3, the cover background is implemented using a simple but festive visual effect — a soft yellow background combined with purple line-drawn stars. The overall color palette enhances the cheerful atmosphere of the celebration.

Elements

- The triangular flags, balloons, and cat on the cover reuse the same drawing functions as those in the main scene.
- Each star is designed in a minimal but elegant way: it consists of four groups of main lines — one vertical line (longest), one horizontal line (medium), and two diagonal lines (slightly shorter).
- Using the `dashRays()` function, a set of dotted radial rays are drawn around each star to create a glowing effect. The stars are animated to change their size over time, creating a twinkling effect.

Title Text

At the bottom of the cover, the main title uses simple scaling to create a perspective effect—objects appear larger when closer. Each character (“2”, “0”, “T”, “H”, heart) is drawn as a pixel grid defined by strings of ’0’ and ’1’. To keep the pixel style while adding depth, lower pixel rows are scaled slightly wider than upper ones, forming an isosceles trapezoid that gives the title a sense of 3D feeling.

Cat Character

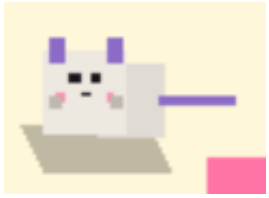
The function drawHero() draws a small cat whose head and body slowly grow and shrink to look like breathing. Its tail also moves left and right in a gentle swing. These movements are made using sine functions.

- `breath` controls the subtle scaling of the head and body.
- `tailSwing` controls the small oscillation of the tail’s angle.

To further illustrate the cat’s animation and pose, the following pair of picture will display tow pose of cat.



(a) Breathing frame.



(b) Tail-swing frame.

Figure 4 Two frames for cat animation (placeholders).

Mouse Interaction

A fish-shaped cursor follows the mouse to make the cover more fun. When the fish gets close to the cat, the cat stops walking and looks at it. If the player moves the cursor away without clicking the left mouse button to “feed” the fish, the cat starts walking again, showing a simple chasing behavior.

4. Scene 2: Campus and Soft Flashlight System

Before diving into the sky and object construction, the following figure sketches the main scene, and scene 2 is only a part of the campus area. The full view (Scene 3) appears gradually as the camera or viewport moves.

Sky

Using the fixed-function OpenGL pipeline (without shaders or textures), the project creates a multi-section gradient sky

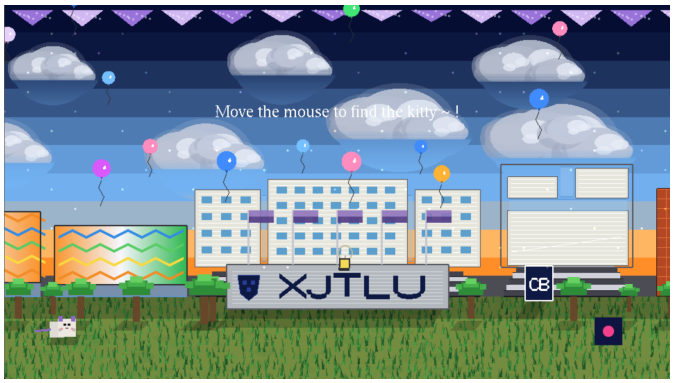


Figure 5 Main scene composition(without black screen.

that shows smooth color changes—from deep night blue to sky blue, golden yellow, warm orange, and red. The sky is drawn with a striped gradient: the visible height is divided into 80–160 thin horizontal strips, and each strip’s top and bottom colors come from a gradient function. This produces a soft layered look.
Although the gradient could be made smoother, I kept a slight “pixel-striped” effect to fit the project’s overall pixel style. Small bright dots are added in the upper half of the screen, and their transparency changes gently over time to look like faint twinkling stars, helping the scene transition naturally into the darker stage.

Clouds

The cloud system is organized hierarchically, from the base function `drawCloudBlob()` to the higher-level `drawCloudCluster()`, allowing clouds to form naturally from small blobs into larger groups.

- Each cloud is composed of multiple semi-transparent ellipses stacked together, forming four layers: shadow, gray body, highlight, and a soft outer rim. Typically, one full cloud combines 3–4 small clusters.
- A flat-cut bottom greatly improves realism. This effect is achieved by drawing a rectangular “mask bar” to trim the lower edge and blend it smoothly with the sky color.
- Clouds of different sizes are arranged in several horizontal rows, creating a natural sense of depth and distance.

The two images below show the effect of the flat-cut cloud base.



(a) Cloud with flat-cut.



(b) Flat-cut base.

Figure 6 Cloud without flat-cut.

Triangular Flags

The flags are drawn in screen-space coordinates, and the projection is reset each frame so they stay at the top of the screen, like a decorative border of the card.
To make the motion look more natural, each flag’s tip moves

up and down using a sine function based on its x-position. Small semi-transparent white dots are added as highlights.



Figure 7 Triangular flags placeholder.

Balloons

Each balloon is made of a round body, a small darker triangular knot, a bright highlight, and a thin line as the string. The balloons float slowly over time, and their vertical movement changes slightly with x-position to make the motion less uniform. Three layouts are used—center, top edge (after camera moves), and sides—with lower density to keep buildings and text clear.

Buildings

The figure below shows the construction of the building section, which mainly includes the shadow, four main building parts, and the stone monument.

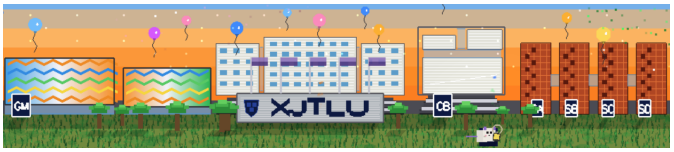


Figure 8 Buildings.

- **Shadows:** Reusable adjustable quadrilateral shadows are used under major structures; transparency and vertical stretch are parameterized.
- **Foundation Building**
  Built from grouped rectangles divided into left, center, and right blocks, with small side wings referring to real photos. A layer of glass strips is added on top.
- **Library**
  The library includes upper-left, upper-right, and lower parts. It mainly uses square shapes with semi-transparent gray rectangles to show glass. The base has several stacked gray rectangles in dark and light tones to create depth and steps.
- **S-Building**
  This building uses reddish-brown blocks with grid lines showing the wall panels. The drawing order is: base color → grid lines → decoration → outer contour. Glass corridors connect the segments, and a main function combines several units into one continuous complex.
- **Gymnasium**
  Similar to the S-building, the gym is made of two joined parts. Each part (`drawGymUnitSimple()`) uses a large rectangle with a horizontal color change to show the glass lower section and tinted upper body. Zigzag lines are used as simple wall decorations. The function `drawGymnasiumSimple()` scales and combines the two parts to form the whole gym.
- **Stone Monument and Nameplate**
  A reusable stone monument system allows color and text changes. The main XJTLU monument (based on the Foundation Building plaza) is larger and more detailed, with blue outline text spelling the university name. On its left, a simplified shield emblem is drawn with vertical, horizontal, and diagonal lines, then outlined again with thicker strokes (the same method is used for the celebration title in Scene 3). Three small dots inside the shield mark the emblem details. Above the monument, small flags in XJTLU colors are displayed, and smaller blue plaques inspired by campus signs are placed in front of the buildings.

Grass and Trees

– **Grass System**
  A solid green base layer is drawn first, followed by dense triangular grass patches. Each patch uses three color tones—dark, mid, and bright—to represent shadow, midtone, and highlight.
  * **Tufts:** 1–4 thin triangles form small grass clusters, placed with a high probability in each area.
  * **Clumps:** Semi-transparent ellipses under the tufts create soft shadows and make the grass appear fuller. Using separate ellipses keeps the scene clean and avoids messy overlapping shadows.

– **Trees**
  Trees use block shapes to match the pixel style. The canopy is made of stacked color blocks from light to dark for shading, and the trunk is a simple vertical rectangle.



Figure 9 Tuft and Clumps.

Dark Mask and Flashlight Effect

After all background elements are drawn, a semi-transparent black layer covers the whole scene.
The stencil buffer is divided into circular zones centered on the flashlight: the outermost area is 0, the rings are 1–4, and the bright center is 5. During rendering, the program draws full-screen black rectangles for each zone, setting different alpha values: darkest outside, lighter toward the center, and none in the middle. This forms a smooth light gradient with soft edges and a bright focus at the center.



Figure 10 Five-ring stencil flashlight (placeholder).

5. Scene 3: Fireworks and Celebration

The following picture stands for the overall celebration scene.

Fireworks System

– Each firework has three stages: **launch**, **burst**, and **fade**, all controlled by a particle system to create dynamic effects.



Figure 11 Celebration scene.

- `launchRocket()` creates and moves rockets upward to random heights defined by `spawnFireworkBurst()`, ensuring explosions occur in the upper half of the screen.
- While rising, `emitTrail(const Rocket& rk)` generates short-lived trail particles with bright colors from the `FW_COLS` palette, forming flickering light dots along the rocket's path.
- At the target height, `emitBurst(cx, cy, n)` produces an explosion by spreading n particles evenly in a circle, each with random velocity, lifetime, and color, with white particles at the center for stronger flashes.
- All particles are stored in the global list `gParticles` and updated each frame by `updateParticles(float dt)`. Particles fade out gradually as their lifetime reaches zero.

6. Text and Hint System

The text display system has two layers:

- A **hint layer** for user prompts.
- A **celebration title layer** for festive text.

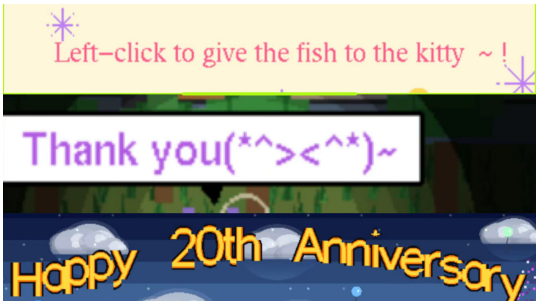


Figure 12 Text system.

Interactive Hint Texts

- `bitmapTextWidth()` calculates the pixel width of a string so that `drawHintTopCenter()` can center it on the screen. The function also supports vertical offset for positioning. Together, they draw normal on-screen hints.
- `drawThankBubbleAboveHero()` displays a floating speech bubble above the main character (e.g., showing "Thank you."). The bubble moves slightly up and down using `sin(gTime)` to create a lively animation.

Celebration Title Texts

- `drawStrokeOutlinedLineCentered_simple()` draws celebration slogans with two layers—an outline and inner text—for a clean, bold look. Then `drawCelebrationBanner()` combines two such lines into a complete banner.

7. Limitations and Improvements

- **Inconsistent naming and redundant functions**
  Some parts of the code contain repeated logic or unclear naming. For example, the main character was initially a human, so many related functions use the prefix `hero`. After changing the character to a cat, the old naming remained, causing confusion and reducing readability.
- **Code structure too heavy**
  The overall code can be simplified. Several initialization or setup functions duplicate similar tasks and could be merged or refactored to improve clarity and maintainability.
- **Lack of window-size adaptability**
  Some rendering methods do not adjust properly to window resizing. When the viewport is reduced, certain buildings or objects may overlap or move out of frame. Future improvements should include adaptive scaling or coordinate normalization to maintain visual consistency across different resolutions.